Benchmark report

Generated: 2026-07-30 12:32 UTC · Julia 1.12.6 · alderlake (32 logical CPUs, 1 Julia thread) · Float128 paths: enabled · Chairmarks 1.3.1

Reference format for per-operation tables: `Binary8p4se` under `(NearestTiesToEven, SatNone)`. Every table names its operand class: the scalar-operation tables appear in four variants — all code points (NaN and ±Inf sampled), NaN excluded, finite-only, and per-operation in-domain — and every other sampled table uses the all-code-points pool, identified in its note. Times are per call; minima first, medians alongside. Methodology per the recorded benchmark doctrine: type-parameterized barriers, untimed setup, specialization preflight.

Core primitives

The decode/compare/step/classify layer plus the projection engine. Operands: all code points — NaN and ±Inf sampled.

operation	min	median	allocs
`decode`	1.6 ns	1.6 ns	0
`order_key`	1.6 ns	1.8 ns	0
`x < y`	1.8 ns	1.8 ns	0
`TotalOrder`	2.1 ns	2.1 ns	0
`Class`	1.6 ns	2.5 ns	0
`NextGreaterThan`	1.8 ns	2.1 ns	0
`project (RNE·SatNone)`	2.2 ns	6.7 ns	0
`project (StochasticA[8], R drawn)`	2.1 ns	7.1 ns	0

Scalar operations

Each arity is measured under four operand classes — safe args, no NaN/Inf, no NaN, and all code points — so operand-class effects (NaN fast rows, ±Inf special rows) are separable.

```
\clearpage
```

Unary (30)

Safe args

Finite operands within each operation's safe domain — the fully unmasked per-operation scalar cost. The argument-restricted ops (Sqrt, RSqrt, Log, Log2, LogOnePlus, Recip, Divide, ArcSin, ArcCos, ArcCosh,

ArcTanh) draw from explicit per-argument safe-domain predicates; every other op uses finite operand tuples whose defined result is not NaN (oracle-derived). Sorted by median.

operation	min	median	allocs
Abs	4.9 ns	6.7 ns	0
Negate	4.8 ns	6.9 ns	0
Recip	7.9 ns	18.0 ns	0
Sqrt	5.7 ns	18.7 ns	0
RSqrt	9.6 ns	20.7 ns	0
Cosh	7.3 ns	21.4 ns	0
Sin	6.2 ns	21.7 ns	0
Cos	7.3 ns	21.7 ns	0
Exp2	7.3 ns	22.4 ns	0
Exp	7.4 ns	22.5 ns	0
ArcSin	5.7 ns	23.2 ns	0
ExpMinusOne	5.3 ns	23.2 ns	0
Tanh	5.8 ns	23.2 ns	0
ArcCos	5.7 ns	24.3 ns	0
ArcSinPi	5.5 ns	25.5 ns	0
Sinh	5.7 ns	25.6 ns	0
Log	5.5 ns	25.7 ns	0
Log2	5.9 ns	26.6 ns	0
Tan	5.3 ns	26.6 ns	0
LogOnePlus	5.4 ns	26.7 ns	0
ArcCosPi	6.1 ns	26.8 ns	0
CosPi	6.9 ns	27.8 ns	0
SinPi	5.6 ns	28.1 ns	0
ArcTan	5.2 ns	28.4 ns	0
ArcTanPi	5.1 ns	31.0 ns	0
ArcTanh	6.2 ns	31.1 ns	0
TanPi	6.2 ns	31.1 ns	0
ArcCosh	5.5 ns	33.3 ns	0

operation	min	median	allocs
Softplus	13.8 ns	33.3 ns	0
ArcSinh	6.2 ns	33.4 ns	0

No NaN, Inf args

Operands exclude NaN and ±Inf; finite datums only (zeros and subnormals kept). Domain-restricted ops still take NaN fast rows on out-of-domain finite operands. Sorted by median.

operation	min	median	allocs
Sqrt	1.9 ns	2.1 ns	0
ArcCosh	2.1 ns	2.1 ns	0
ArcSinPi	1.9 ns	2.2 ns	0
Abs	4.3 ns	6.7 ns	0
Negate	4.8 ns	6.9 ns	0
Log2	2.1 ns	8.6 ns	0
RSqrt	2.1 ns	9.5 ns	0
Recip	1.9 ns	17.8 ns	0
Cos	7.1 ns	21.2 ns	0
Sin	5.4 ns	21.5 ns	0
Cosh	7.0 ns	21.6 ns	0
Exp2	7.3 ns	22.3 ns	0
Exp	7.0 ns	22.4 ns	0
ExpMinusOne	4.5 ns	22.8 ns	0
ArcSin	2.3 ns	22.8 ns	0
Tanh	5.7 ns	23.1 ns	0
ArcCos	2.5 ns	23.8 ns	0
Log	2.3 ns	25.3 ns	0
Sinh	5.1 ns	25.6 ns	0
LogOnePlus	1.9 ns	26.3 ns	0
ArcCosPi	2.1 ns	26.5 ns	0
Tan	6.1 ns	26.6 ns	0
CosPi	6.4 ns	27.4 ns	0

operation	min	median	allocs
SinPi	5.2 ns	27.7 ns	0
ArcTan	5.5 ns	28.3 ns	0
ArcTanh	2.1 ns	28.7 ns	0
TanPi	5.3 ns	30.8 ns	0
ArcTanPi	4.8 ns	30.8 ns	0
Softplus	14.3 ns	33.5 ns	0
ArcSinh	5.0 ns	33.6 ns	0

No NaN args

Operands exclude the NaN code point; ±Inf and every finite datum are sampled. Sorted by median.

operation	min	median	allocs
ArcCosh	2.1 ns	2.1 ns	0
RSqrt	2.1 ns	2.2 ns	0
Log	2.3 ns	2.5 ns	0
ArcSin	2.5 ns	2.6 ns	0
Sqrt	1.9 ns	5.1 ns	0
Abs	4.4 ns	6.7 ns	0
Negate	4.4 ns	6.8 ns	0
ArcSinPi	2.0 ns	7.3 ns	0
Log2	2.1 ns	8.6 ns	0
Recip	1.9 ns	17.8 ns	0
Cosh	5.0 ns	21.2 ns	0
Sin	2.1 ns	21.4 ns	0
Cos	1.9 ns	21.5 ns	0
Exp2	5.5 ns	22.3 ns	0
Exp	5.0 ns	22.3 ns	0
Tanh	5.7 ns	23.0 ns	0
ExpMinusOne	4.6 ns	23.1 ns	0
ArcCos	2.6 ns	23.8 ns	0
Sinh	5.1 ns	25.6 ns	0

operation	min	median	allocs
LogOnePlus	1.9 ns	26.2 ns	0
ArcCosPi	2.1 ns	26.6 ns	0
Tan	2.0 ns	27.2 ns	0
CosPi	2.5 ns	27.5 ns	0
SinPi	2.1 ns	27.7 ns	0
ArcTan	5.7 ns	28.3 ns	0
ArcTanh	2.1 ns	28.6 ns	0
ArcTanPi	4.7 ns	30.8 ns	0
TanPi	2.1 ns	30.8 ns	0
Softplus	4.8 ns	33.5 ns	0
ArcSinh	4.5 ns	33.6 ns	0

All code points

Operands drawn uniformly over ALL code points — NaN and ±Inf are sampled; medians of domain-restricted ops are diluted by instant NaN rows. Sorted by median. Transcendental rows mix special-row fast returns with enclosure-path evaluations, so these are *scalar-path* costs; bulk unary work routes through 256-byte tables (see Array kernels).

operation	min	median	allocs
Sqrt	1.9 ns	2.0 ns	0
ArcCosh	2.1 ns	2.1 ns	0
ArcSinPi	1.9 ns	2.1 ns	0
RSqrt	2.1 ns	2.1 ns	0
Log	2.3 ns	3.8 ns	0
Log2	2.1 ns	5.5 ns	0
ArcSin	2.1 ns	5.8 ns	0
ArcTanh	2.1 ns	6.6 ns	0
Abs	2.0 ns	6.7 ns	0
Negate	2.1 ns	6.8 ns	0
Recip	1.8 ns	17.8 ns	0
Cosh	2.3 ns	21.0 ns	0
Cos	1.9 ns	21.3 ns	0

operation	min	median	allocs
Sin	2.1 ns	21.5 ns	0
Exp2	2.1 ns	22.3 ns	0
Exp	2.0 ns	22.3 ns	0
ExpMinusOne	2.0 ns	22.7 ns	0
Tanh	2.3 ns	23.1 ns	0
ArcCos	2.3 ns	23.7 ns	0
Sinh	2.3 ns	25.5 ns	0
LogOnePlus	1.8 ns	26.4 ns	0
Tan	2.0 ns	26.5 ns	0
ArcCosPi	1.9 ns	26.6 ns	0
CosPi	2.3 ns	27.5 ns	0
SinPi	2.0 ns	27.7 ns	0
ArcTan	2.2 ns	28.2 ns	0
ArcTanPi	2.1 ns	30.8 ns	0
TanPi	1.9 ns	30.8 ns	0
Softplus	2.0 ns	33.5 ns	0
ArcSinh	2.1 ns	33.6 ns	0

```
\clearpage
```

Binary (18)

Safe args

Finite operands within each operation's safe domain — the fully unmasked per-operation scalar cost. The argument-restricted ops (Sqrt, RSqrt, Log, Log2, LogOnePlus, Recip, Divide, ArcSin, ArcCos, ArcCosh, ArcTanh) draw from explicit per-argument safe-domain predicates; every other op uses finite operand tuples whose defined result is not NaN (oracle-derived). Sorted by median.

operation	min	median	allocs
MinimumMagnitude	4.9 ns	7.1 ns	0
MaximumMagnitude	6.9 ns	7.1 ns	0
CopySign	4.8 ns	7.2 ns	0

operation	min	median	allocs
Maximum	4.5 ns	7.2 ns	0
Minimum	4.6 ns	7.2 ns	0
MaximumFinite	4.8 ns	7.2 ns	0
MinimumFinite	4.9 ns	7.3 ns	0
MinimumNumber	4.8 ns	7.3 ns	0
MaximumNumber	5.0 ns	7.3 ns	0
MaximumMagnitudeNumber	7.1 ns	7.3 ns	0
MinimumMagnitudeNumber	4.8 ns	7.3 ns	0
Multiply	4.9 ns	7.4 ns	0
Add	7.1 ns	9.0 ns	0
Subtract	7.4 ns	9.2 ns	0
Divide	6.4 ns	18.3 ns	0
Hypot	7.8 ns	28.0 ns	0
ArcTan2	8.0 ns	32.7 ns	0
ArcTan2Pi	7.2 ns	35.3 ns	0

No NaN, Inf args

Operands exclude NaN and ±Inf; finite datums only (zeros and subnormals kept). Domain-restricted ops still take NaN fast rows on out-of-domain finite operands. Sorted by median.

operation	min	median	allocs
MinimumMagnitude	4.8 ns	7.1 ns	0
MaximumMagnitude	6.9 ns	7.1 ns	0
Minimum	4.7 ns	7.2 ns	0
Maximum	4.6 ns	7.2 ns	0
CopySign	4.7 ns	7.2 ns	0
MinimumFinite	5.0 ns	7.2 ns	0
MaximumFinite	5.1 ns	7.2 ns	0
MinimumNumber	6.1 ns	7.3 ns	0
MaximumNumber	4.8 ns	7.3 ns	0
MinimumMagnitudeNumber	4.8 ns	7.3 ns	0

operation	min	median	allocs
MaximumMagnitudeNumber	7.2 ns	7.3 ns	0
Multiply	4.9 ns	7.4 ns	0
Add	7.1 ns	9.0 ns	0
Subtract	7.9 ns	9.5 ns	0
Divide	2.6 ns	18.3 ns	0
Hypot	8.2 ns	28.0 ns	0
ArcTan2	9.0 ns	32.7 ns	0
ArcTan2Pi	7.1 ns	35.4 ns	0

No NaN args

Operands exclude the NaN code point; ±Inf and every finite datum are sampled. Sorted by median.

operation	min	median	allocs
MinimumMagnitude	4.6 ns	7.1 ns	0
MaximumMagnitude	5.0 ns	7.1 ns	0
CopySign	5.0 ns	7.2 ns	0
Minimum	4.6 ns	7.2 ns	0
Maximum	4.6 ns	7.2 ns	0
MaximumFinite	5.0 ns	7.2 ns	0
MinimumFinite	5.0 ns	7.2 ns	0
MinimumNumber	5.1 ns	7.3 ns	0
MaximumNumber	4.6 ns	7.3 ns	0
MinimumMagnitudeNumber	4.6 ns	7.3 ns	0
MaximumMagnitudeNumber	5.1 ns	7.3 ns	0
Multiply	5.0 ns	7.4 ns	0
Add	6.3 ns	9.0 ns	0
Subtract	7.2 ns	9.5 ns	0
Divide	2.6 ns	18.3 ns	0
Hypot	6.1 ns	28.0 ns	0
ArcTan2	7.1 ns	32.7 ns	0
ArcTan2Pi	6.7 ns	35.4 ns	0

All code points

Operands drawn uniformly over ALL code points — NaN and ±Inf are sampled; medians of domain-restricted ops are diluted by instant NaN rows. Sorted by median.

operation	min	median	allocs
MinimumMagnitude	2.1 ns	7.1 ns	0
MaximumMagnitude	1.9 ns	7.1 ns	0
CopySign	2.1 ns	7.2 ns	0
Minimum	2.1 ns	7.2 ns	0
Maximum	1.8 ns	7.2 ns	0
MaximumFinite	5.6 ns	7.2 ns	0
MinimumFinite	5.0 ns	7.2 ns	0
MinimumNumber	5.1 ns	7.3 ns	0
MaximumNumber	4.6 ns	7.3 ns	0
MinimumMagnitudeNumber	4.6 ns	7.3 ns	0
MaximumMagnitudeNumber	5.0 ns	7.4 ns	0
Multiply	2.1 ns	7.4 ns	0
Add	3.7 ns	9.0 ns	0
Subtract	4.6 ns	9.5 ns	0
Divide	2.5 ns	18.3 ns	0
Hypot	3.0 ns	28.0 ns	0
ArcTan2	4.6 ns	32.7 ns	0
ArcTan2Pi	3.8 ns	35.3 ns	0

Ternary (3)

Safe args

Finite operands within each operation's safe domain — the fully unmasked per-operation scalar cost. The argument-restricted ops (Sqrt, RSqrt, Log, Log2, LogOnePlus, Recip, Divide, ArcSin, ArcCos, ArcCosh, ArcTanh) draw from explicit per-argument safe-domain predicates; every other op uses finite operand tuples whose defined result is not NaN (oracle-derived). Sorted by median.

operation	min	median	allocs
Clamp	5.1 ns	8.0 ns	0
FMA	8.1 ns	9.6 ns	0

operation	min	median	allocs
FAA	8.3 ns	9.9 ns	0

No NaN, Inf args

Operands exclude NaN and ±Inf; finite datums only (zeros and subnormals kept). Domain-restricted ops still take NaN fast rows on out-of-domain finite operands. Sorted by median.

operation	min	median	allocs
Clamp	5.5 ns	8.0 ns	0
FMA	8.3 ns	9.7 ns	0
FAA	8.1 ns	9.7 ns	0

No NaN args

Operands exclude the NaN code point; ±Inf and every finite datum are sampled. Sorted by median.

operation	min	median	allocs
Clamp	5.2 ns	8.0 ns	0
FMA	6.9 ns	9.7 ns	0
FAA	7.1 ns	9.7 ns	0

All code points

Operands drawn uniformly over ALL code points — NaN and ±Inf are sampled; medians of domain-restricted ops are diluted by instant NaN rows. Sorted by median.

operation	min	median	allocs
Clamp	2.1 ns	8.0 ns	0
FMA	3.7 ns	9.7 ns	0
FAA	4.1 ns	9.7 ns	0

Sensitivity studies

How the scalar costs move with the format and with the projection specification.

Format sensitivity

Same three binary ops across formats; `Binary8p1uf` exercises the wide-exponent-spread escalations, small-K formats the tiny-table regime. Operands: all code points — NaN and ±Inf sampled.

operation	min	median	allocs
Add⟨Binary8p4se⟩	3.7 ns	9.0 ns	0
Divide⟨Binary8p4se⟩	2.5 ns	18.3 ns	0
Multiply⟨Binary8p4se⟩	2.1 ns	7.4 ns	0
Add⟨Binary8p3sf⟩	3.9 ns	8.7 ns	0
Divide⟨Binary8p3sf⟩	2.6 ns	16.4 ns	0
Multiply⟨Binary8p3sf⟩	2.1 ns	7.2 ns	0
Add⟨Binary8p1uf⟩	4.7 ns	74.0 ns	0
Divide⟨Binary8p1uf⟩	2.6 ns	8.4 ns	0
Multiply⟨Binary8p1uf⟩	2.1 ns	6.9 ns	0
Add⟨Binary5p2se⟩	4.0 ns	9.1 ns	0
Divide⟨Binary5p2se⟩	2.5 ns	9.0 ns	0
Multiply⟨Binary5p2se⟩	1.9 ns	7.4 ns	0
Add⟨Binary3p1se⟩	4.0 ns	7.6 ns	0
Divide⟨Binary3p1se⟩	2.6 ns	6.3 ns	0
Multiply⟨Binary3p1se⟩	2.1 ns	5.9 ns	0

Projection by rounding/saturation mode

`project(Binary8p4se, ρ, x)` over the mode vocabulary (stochastic budgets N = 8). Operands: all code points — NaN and ±Inf sampled.

operation	min	median	allocs
NearestTiesToEven	2.0 ns	6.7 ns	0
NearestTiesToAway	2.8 ns	7.7 ns	0
TowardPositive	3.0 ns	6.0 ns	0
TowardNegative	2.6 ns	5.9 ns	0
TowardZero	2.5 ns	5.3 ns	0
ToOdd	2.0 ns	6.6 ns	0
StochasticA[8]	2.0 ns	7.0 ns	0
StochasticB[8]	2.5 ns	7.1 ns	0
StochasticC[8]	2.3 ns	7.1 ns	0
RNE · SatFinite	2.1 ns	7.6 ns	0

operation	min	median	allocs
`RNE · SatPropagate`	2.1 ns	7.2 ns	0

Kernels and storage

Array-shaped work: the table-gather and compute kernels, the ternary bitwidth policy, sorting, table builds, blocks, and packed/converted storage.

Array kernels (vmap)

Warm caches: table specializations prebuilt, so table rows measure the gather; scalar-loop rows measure the full compute pipeline per element. The ternary row here is `Binary8p4se` (K=8, always the compute path); see the next section for how the ternary bitwidth policy behaves across K. Operands: all code points — NaN and ±Inf sampled.

operation	min	median	allocs	per element
`vmap unary (table gather), n=65536`	8.56 µs	8.71 µs	0	0.13 ns/elem — 7.52 Gelem/s
`vmap binary (table gather), n=65536`	16.45 µs	16.81 µs	0	0.26 ns/elem — 3.9 Gelem/s
`vmap ternary (scalar loop), n=65536`	1.01 ms	1.02 ms	0	15.54 ns/elem — 0.06 Gelem/s
`vmap binary stochastic (scalar loop), n=65536`	646.57 µs	655.78 µs	0	10.01 ns/elem — 0.1 Gelem/s
`vmap unary through PackedVector, n=65536`	76.97 µs	90.29 µs	773	1.38 ns/elem — 0.73 Gelem/s

Ternary bitwidth tiers (FMA/FAA)

`FMA`/`FAA`/`Clamp` are total functions on `2^(K1+K2+K3)` code points, but that count spans 512 B (K=3) to 16 MiB (K=8), so the array kernel tables small operand formats eagerly, tables mid-size ones adaptively (after enough elements amortize the build; not shown here — see the adaptive-cache gate in `test/ternary_opt.jl`), and always runs the scalar compute kernel at K=8, threaded above a size cutoff when `Threads.nthreads() > 1`. Each tier's optimized row is paired with a scalar-loop baseline (policy Refs forced off around the measurement, restored after) so the win is visible per tier; this process has 1 Julia thread. No threaded/sequential comparison below — rerun with `julia -t N` (N > 1) to see it. Same reference format, ρ, and operand pool discipline as Array kernels above.

operation	min	median	allocs	per element
`FMA K=4 (eager table), n=65536`	19.58 µs	20.46 µs	0	0.31 ns/elem — 3.2 Gelem/s
`FMA K=4 (eager table), scalar-loop baseline, n=65536`	967.88 µs	974.03 µs	0	14.86 ns/elem — 0.07 Gelem/s

operation	min	median	allocs	per element
`FMA K=6 (eager table), n=65536`	24.03 µs	24.34 µs	0	0.37 ns/elem — 2.69 Gelem/s
`FMA K=6 (eager table), scalar-loop baseline, n=65536`	1.0 ms	1.01 ms	0	15.46 ns/elem — 0.06 Gelem/s
`FMA K=8 (compute), n=65536`	990.69 µs	1.0 ms	0	15.31 ns/elem — 0.07 Gelem/s
`FMA K=8 (compute), scalar-loop baseline, n=65536`	990.11 µs	1.0 ms	0	15.27 ns/elem — 0.07 Gelem/s

Sorting (64 K values)

Counting sort is installed as the default algorithm for `Binary` vectors. Operands: all code points — NaN and ±Inf sampled.

operation	min	median	allocs
`sort! (counting sort via defalg), n=65536`	155.36 µs	157.4 µs	2
`sort! (stock comparison sort), n=65536`	1.36 ms	1.37 ms	3
`sort! rev=true (counting sort), n=65536`	155.48 µs	157.95 µs	2

Table builds (oracle + projection, Float128-first)

Cold cache per sample (`empty_tables!` in untimed setup); JIT pre-warmed. The warm-hit column is the steady-state cost of `get_table` when the specialization is already cached (min / median). Table entries enumerate every code point by construction (NaN and ±Inf included).

operation	min	median	allocs	warm hit
`Exp⟨8p4se⟩ (256 entries)`	30.31 µs	31.16 µs	257	65.8 ns / 67.4 ns
`Tanh⟨8p4se⟩ (256 entries)`	29.49 µs	30.93 µs	257	65.4 ns / 67.5 ns
`Add⟨8p4se×8p4se⟩ (64 K entries)`	13.77 ms	13.93 ms	458754	65.0 ns / 66.6 ns
`Divide⟨8p4se×8p4se⟩ (64 K)`	14.02 ms	14.14 ms	458754	65.1 ns / 66.6 ns
`Add⟨8p1uf×8p1uf⟩ (64 K, wide-spread)`	27.08 ms	29.5 ms	995304	65.8 ns / 67.8 ns

Block and scaled operations

Elements `Binary8p4se`, scales `Binary8p1uf`, B = 32. Operands: all code points — NaN and ±Inf sampled.

operation	min	median	allocs	per lane
`BlockAdd (B=32)`	963.7 ns	1.17 µs	35	36.65 ns/lane
`BlockDotProduct → 8p4se (B=32)`	211.3 ns	265.3 ns	3	8.29 ns/lane

operation	min	median	allocs	per lane
`BlockReduceAdd → 8p4se (B=32)`	43.5 ns	489.4 ns	0	15.29 ns/lane
`ConvertToBlockMaxAbsFinite (B=32)`	3.79 µs	3.95 µs	111	123.3 ns/lane

Conversions and packed storage

Operands: all code points — NaN and ±Inf sampled.

operation	min	median	allocs	per element
`T(::UInt8) code-point constructor`	1.8 ns	1.8 ns	0	
`rawvalue (unchecked kernel route)`	1.8 ns	1.8 ns	0	
`T(::Float64) numeric constructor (projects)`	3.2 ns	7.7 ns	0	
`Convert 8p4se → 8p3se (scalar)`	2.1 ns	6.7 ns	0	
`Float64 → 8p4se (project)`	2.0 ns	6.7 ns	0	
`PackedVector pack, n=65536`	33.47 µs	35.43 µs	3	0.54 ns/elem
`PackedVector unpack (collect), n=65536`	28.86 µs	29.05 µs	3	0.44 ns/elem

All numbers from this machine/run; absolute values vary by host. Regenerate with `julia --project=benchmarking benchmarking/benchmarking.jl benchmarking/benchmark_report.md`.